Understanding the Sargantana Microarchitecture

An RTL-Oriented Study of the Sargantana RISC-V Processor

Author
Muhammad Waleed Akram

Repository: `https://github.com/bsc-loca/sargantana`

July 26, 2026

Contents

Chapter 1

Introduction

1.1 About Sargantana

Sargantana is an open-source, Linux-capable, high-performance RISC-V processor developed by the Barcelona Supercomputing Center (BSC). It is designed as an out-of-order superscalar processor capable of executing multiple instructions simultaneously while maintaining precise architectural state. Unlike simple in-order RISC-V processors commonly used for educational purposes, Sargantana incorporates modern microarchitectural techniques such as speculative execution, branch prediction, register renaming, out-of-order execution, and in-order instruction retirement.

1.2 Key Features

Some of the primary architectural features of the Sargantana processor include:

- Open-source RISC-V implementation.
- Linux-capable processor architecture.
- Out-of-order instruction execution.
- Superscalar instruction issue.
- Multi-stage pipelined datapath.
- Dynamic branch prediction.
- Speculative instruction execution.
- Register renaming using physical registers.
- In-order instruction commitment.
- Modular RTL organization.
- Extensive architectural documentation and design specifications.

The processor serves both as a research platform for computer architecture and as a foundation for exploring advanced hardware features within the RISC-V ecosystem. Owing to its modular RTL implementation and comprehensive documentation, Sargantana is well suited for architectural experimentation and ISA extension development.

1.3 Objectives of this Study

The primary objective of this document is to develop a comprehensive understanding of the Sargantana processor architecture through a systematic study of its RTL implementation and accompanying documentation. The specific objectives include:

- Study the overall Sargantana microarchitecture.
- Understand the functionality of each pipeline stage.
- Analyze the interaction between the datapath and control logic.
- Examine the major RTL modules and their hierarchy.
- Relate architectural concepts to their corresponding RTL implementation.
- Identify the processor components involved in instruction fetch, execution, and retirement.
- Understand the processor's recovery and exception handling mechanisms.
- Identify potential integration points for the RISC-V Control-Flow Integrity extensions.

1.4 Repository Organization

Directory / File	**Description**
rtl/	Core RTL implementation
rtl/top_drac.sv	Top-level module instantiating datapath, CSR, MMU, HPM
rtl/datapath/	Pipeline stages and execution units
rtl/datapath/rtl/datapath.sv	Top-level datapath implementation
rtl/datapath/rtl/if_stage_1/	Instruction fetch + branch prediction
rtl/datapath/rtl/if_stage_2/	Fetch buffer
rtl/datapath/rtl/id_stage/	Instruction decode
rtl/datapath/rtl/ir_stage/	Instruction rename (register renaming, free list)
rtl/datapath/rtl/rr_stage/	Register read
rtl/datapath/rtl/exe_stage/	Execution with ALU, FPU, SIMD, load/store/div units
rtl/datapath/rtl/wb_stage/	Write-back and graduation
rtl/control_unit/	Control logic
rtl/control_unit/rtl/control_unit.sv	Top-level control unit module
rtl/csr/	Control/Status Registers and HPM counters (submodule) → empty
rtl/mmu/	Memory Management Unit (submodule) → empty
rtl/common_cells/	PULP platform utilities (lzc, fifo, arbiter, etc.) → empty
`doc/`	Documentation
`doc/specifications/`	Detailed module specifications (LaTeX)

Directory / File	Description
doc/riscv-spec.pdf	RISC-V ISA specification
doc/riscv-privileged.pdf	RISC-V privileged specification
doc/sargantana_pipeline.svg	Pipeline diagram
scripts/	CI and verification utilities
scripts/veri_lint.sh	Verilator linting
scripts/questa_ci.sh	Questa ModelSim simulation
scripts/spyglass_ci.sh	Static analysis (Spyglass)
scripts/parse-yaml-fpnew.py	FPU configuration
tb/	Testbenches
tb/tb_top/	Top-level test vectors (RISC-V vector assembly)
includes/	Shared packages
includes/drac_pkg.sv	Configuration and types
includes/riscv_pkg.sv	RISC-V constants
includes/def_pkg.sv	General definitions

Chapter 2

Sargantana Pipeline Overview

2.1 Pipeline Architecture

The processor uses register renaming, speculative execution, branch prediction, instruction queues, multiple execution units, and an in-order commit mechanism. These features allow instructions to execute out of program order while preserving the correct architectural state at retirement.

The complete processor pipeline is illustrated in Figure 2.1.

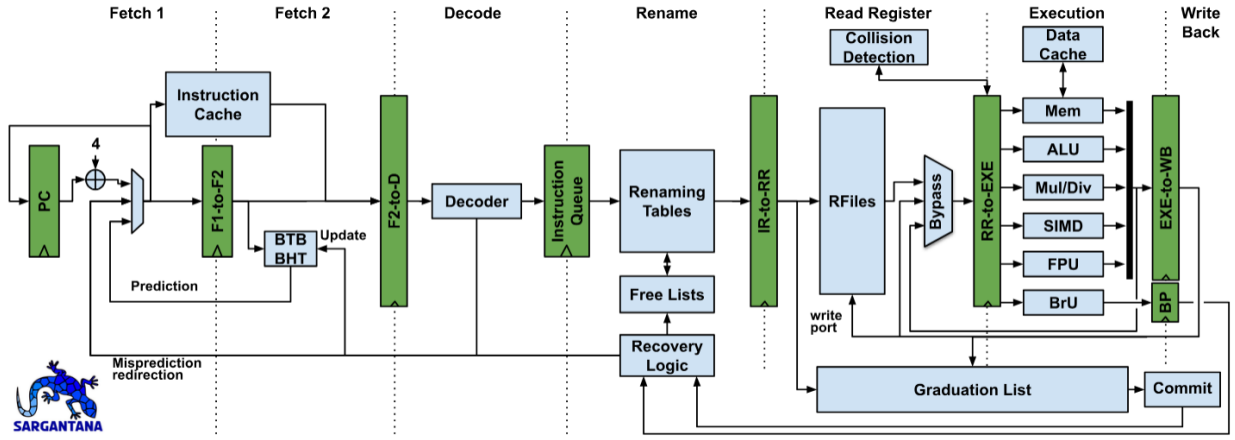

Figure 2.1: Sargantana complete processor pipeline

The major stages of the pipeline are:

- Fetch 1
- Fetch 2
- Decode
- Rename
- Register Read
- Execution

- Write Back
- Commit

Each stage performs a specific task and communicates with adjacent stages through dedicated pipeline registers.

2.2 Pipeline Stages

The Sargantana pipeline consists of eight primary stages, each responsible for a particular phase of instruction execution.

Fetch 1

The Fetch 1 stage maintains the Program Counter (PC) and communicates with the instruction cache to request instructions. Branch prediction hardware, including the Branch Target Buffer (BTB) and Branch History Table (BHT), is also consulted to predict future control flow before branch instructions are resolved.

Fetch 2

The second fetch stage receives instructions from the instruction cache and forwards them to the decode stage through the F2-to-D pipeline register. During this stage, prediction information generated in Fetch 1 accompanies the fetched instructions.

Decode

The decode stage interprets the binary instruction encoding and determines:

- instruction type
- source registers
- destination register
- immediate values
- execution unit selection
- control signals

The decoded instruction is then inserted into the instruction queue for subsequent processing.

Rename

One of the defining characteristics of Sargantana is its register renaming stage. Architectural registers are mapped onto physical registers to eliminate false dependencies such as Write-After-Write (WAW) and Write-After-Read (WAR) hazards.

This stage also interacts with:

- Renaming Tables
- Free Lists

- Recovery Logic

to manage physical register allocation and pipeline recovery following branch mispredictions or exceptions.

Register Read

After renaming, source operands are read from the physical register file. If the required operands have recently been produced but not yet written back, bypass logic forwards the values directly from later pipeline stages to reduce execution latency.

Execution

Instructions are dispatched to specialized execution units depending on their operation. These include:

- Arithmetic Logic Unit (ALU)
- Branch Unit (BrU)
- Load/Store Unit (Memory)
- Multiply/Divide Unit
- Floating Point Unit (FPU)
- SIMD Unit

Each functional unit operates independently, allowing several instructions to execute concurrently.

Write Back

Upon completion of execution, results are written back into the physical register file. These results become available for dependent instructions through both register reads and forwarding paths.

Commit

Although instructions execute out-of-order, they retire strictly in program order. The commit stage updates the architectural state only after ensuring that all previous instructions have completed successfully. This guarantees precise exceptions and preserves program correctness.

2.3 Instruction Flow through the Pipeline

Figure 2.1 illustrates how instructions travel through the processor pipeline.

Unlike an in-order processor, instructions may execute as soon as their operands become available rather than waiting for older instructions to finish execution. However, all instructions commit in the original program order, ensuring correct architectural behavior.

Chapter 3

Top-Level Datapath

3.1 Introduction

The datapath forms the computational backbone of the Sargantana processor. It interconnects every major pipeline stage, coordinates the movement of instructions and operands, and interfaces with all execution units. While the control logic determines *what* operation must be performed, the datapath determines *where* information flows and *how* the processor state is updated.

At the highest level, the datapath receives instructions from the instruction fetch subsystem, processes them through decoding, register renaming, operand reading, execution, and finally commits the architectural state once execution completes successfully.

The complete organization of the datapath is illustrated in Figure 3.1.

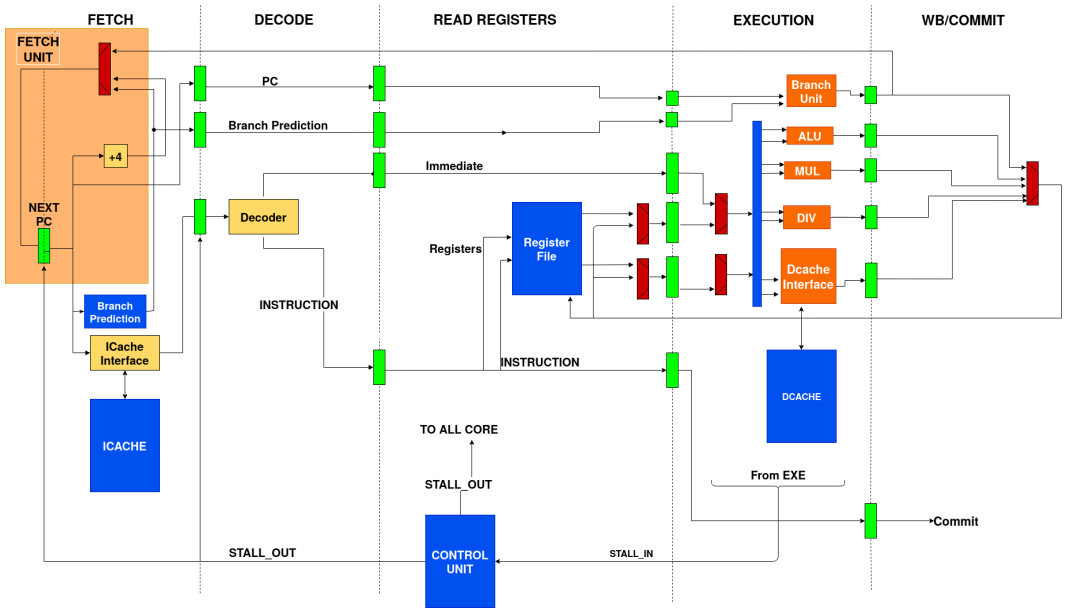

Figure 3.1: Top-level organization of the Sargantana datapath

3.2 Top-Level Interfaces

The top-level datapath module serves as the central connection point between the frontend, backend, execution units, memory subsystem, and commit logic.

Its primary interfaces include:

Instruction Interface

Receives decoded instructions originating from the frontend pipeline. Connected modules include:

- Decoder
- Instruction Queue

Register Interface

Communicates with the physical register file to obtain source operands and write execution results. Connected modules include:

- Register File
- Rename Logic
- Bypass Network

Execution Interface

Dispatches instructions to the appropriate execution unit based on the decoded operation. Supported execution units include:

- ALU
- Branch Unit
- Load/Store Unit
- Multiply/Divide Unit
- SIMD Unit
- Floating Point Unit

Commit Interface

Transfers completed instructions to the Graduation List and Commit stage where architectural state is updated.

Recovery Interface

Provides rollback support whenever

- branch prediction fails,
- exceptions occur,
- interrupts arrive.

Recovery information is exchanged with the Rename stage and Commit logic to restore a precise architectural state.

3.3 Internal Architecture

Internally, the datapath acts as the interconnection fabric of the processor. Instead of implementing computation directly, it instantiates and connects all major pipeline components.

The internal architecture consists of:

- Pipeline Registers
- Instruction Queue
- Rename Tables
- Physical Register File
- Bypass Network
- Functional Units
- Commit Logic

3.4 Pipeline Registers

To achieve high throughput, Sargantana separates adjacent pipeline stages using dedicated pipeline registers.

Important pipeline registers include:

- F1-to-F2
- F2-to-D
- IR-to-RR
- RR-to-EXE
- EXE-to-WB

These registers isolate combinational logic between stages, allowing multiple instructions to progress simultaneously through different portions of the processor.

Chapter 4

Instruction Fetch Unit

4.1 Overview

The Instruction Fetch Unit (IFU) is responsible for supplying a continuous stream of instructions to the processor pipeline. Its primary responsibilities include maintaining the Program Counter (PC), accessing the instruction cache, performing branch prediction, and delivering fetched instructions to the decode stage.

The fetch subsystem is divided into two pipeline stages:

- Fetch Stage 1 (F1)
- Fetch Stage 2 (F2)

This separation enables the processor to overlap instruction fetching with cache access and branch prediction, thereby improving instruction throughput.

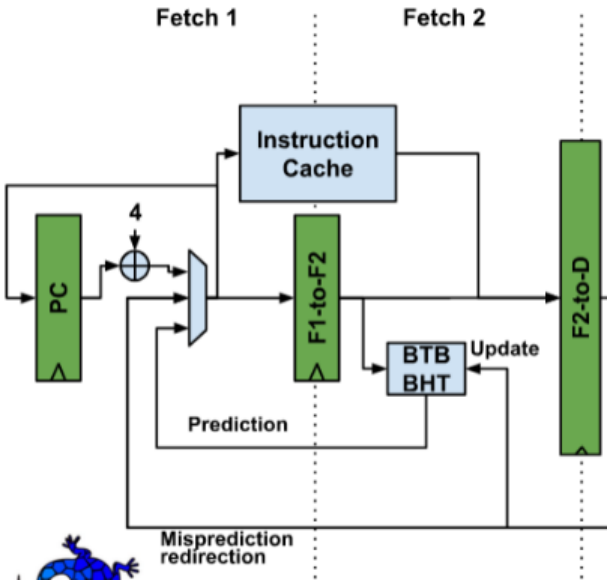

Figure 4.1: Fetch Stage 1 and Fetch Stage 2 organization

4.2 Program Counter

The Program Counter (PC) identifies the address of the next instruction to be fetched. During normal execution, the PC advances sequentially by the instruction width. However, the fetch stage may also redirect the PC based on:

- Branch prediction
- Jump instructions
- Exceptions
- Interrupts
- Pipeline recovery after branch misprediction

Whenever a branch prediction is later determined to be incorrect, the recovery logic updates the PC with the correct target address, and instruction fetching resumes from the corrected location.

4.3 Fetch Stage 1 (F1)

Fetch Stage 1 initiates instruction fetching. Its primary responsibilities include:

- Maintaining the Program Counter
- Accessing the Branch Target Buffer (BTB)
- Reading the Branch History Table (BHT)
- Generating prediction information
- Sending instruction addresses to the instruction cache

This stage attempts to predict future control flow before the instruction has even been decoded, thereby reducing branch penalties.

4.4 Instruction Cache

The Instruction Cache stores recently accessed instructions to reduce memory access latency. When Fetch Stage 1 supplies an instruction address, the cache checks whether the corresponding instruction is already available.

- **Cache Hit:** The instruction is returned immediately.
- **Cache Miss:** The memory hierarchy is accessed to retrieve the required cache line.

The instruction cache therefore plays a crucial role in sustaining a high fetch bandwidth and minimizing pipeline stalls.

4.5 Fetch Stage 2 (F2)

Fetch Stage 2 receives the fetched instruction and associated prediction information from the instruction cache.

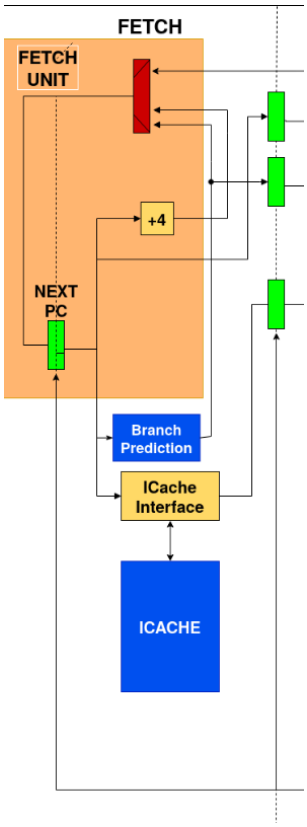

Figure 4.2: Internal detail of the Fetch Unit: PC, +4 adder, ICache interface, and branch prediction

Its responsibilities include:

- Buffering fetched instructions
- Packaging instruction metadata
- Forwarding branch prediction information
- Passing instructions to the Decode stage via the F2-to-D pipeline register

At this point, the instruction has not yet been decoded; it remains in its original binary form.

Chapter 5

Branch Prediction Subsystem

5.1 Overview

One of the biggest performance bottlenecks in modern processors is the presence of branch instructions. Every conditional branch introduces uncertainty because the processor does not immediately know which instruction should be fetched next. Waiting until the branch is resolved would stall the pipeline and significantly reduce performance.

To avoid these stalls, Sargantana performs **speculative execution** by predicting the outcome of branches before they are executed. If the prediction is correct, instruction execution continues without interruption. If the prediction is incorrect, speculative instructions are discarded and the processor restarts from the correct target address.

Unlike a simple five-stage processor, Sargantana performs branch prediction during the **Fetch-1 (F1)** stage so that the following stages remain continuously supplied with instructions.

RTL References

- *rtl/datapath/rtl/if_stage_1/rtl/if_stage_1.sv*
- *rtl/datapath/rtl/if_stage_1/rtl/branch_predictor.sv*

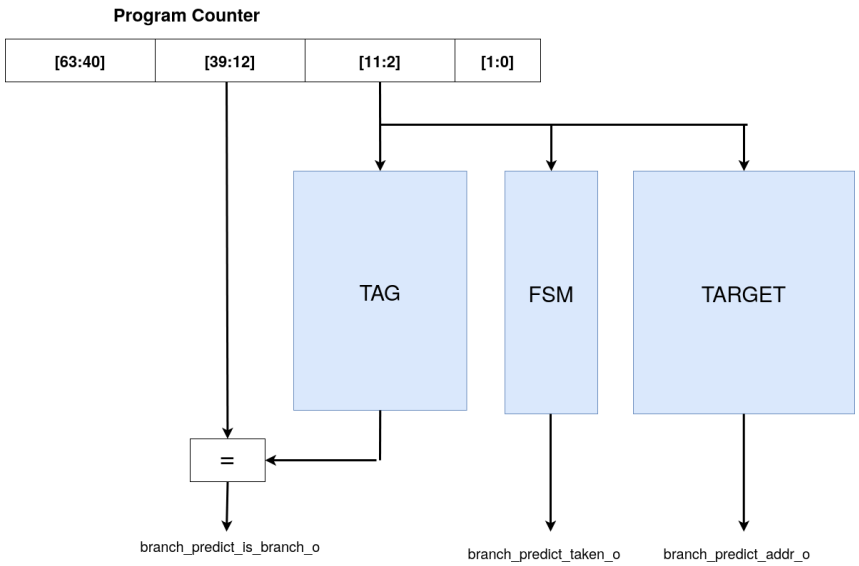

Figure 5.1: Program Counter fields feeding the TAG, FSM, and TARGET structures of the branch predictor

5.2 Speculative Execution

Speculative execution allows instructions located after a predicted branch to begin moving through the pipeline before the branch outcome is known.

The branch predictor selects one of two possible next PCs:

- Sequential PC (PC + 4)
- Predicted branch target

The Fetch stage immediately begins fetching instructions from the selected address. If the prediction later proves correct, execution continues normally. If incorrect, the processor:

- flushes incorrect pipeline entries,
- restores architectural state,
- redirects the PC to the correct target.

This recovery mechanism minimizes branch penalties while maintaining correctness.

RTL References

- *rtl/datapath/rtl/if_stage_1/rtl/if_stage_1.sv*
- *rtl/control_unit/rtl/control_unit.sv*

5.3 Branch Target Buffer (BTB)

The Branch Target Buffer stores previously encountered branch instructions together with their destination addresses.

Whenever a new PC enters Fetch-1, the BTB is searched. If a matching entry exists, the BTB immediately supplies the predicted destination address instead of waiting for the branch instruction to execute.

Therefore, BTB answers the question:

"Where should execution go?"

Its primary purpose is reducing control hazards caused by indirect and conditional branches. Within Sargantana, BTB functionality is managed by the branch prediction subsystem inside the Fetch-1 stage.

RTL References

- *rtl/datapath/rtl/if_stage_1/rtl/branch_predictor.sv*

5.4 Branch History Table (BHT)

Predicting only the target address is insufficient because the processor must also determine whether the branch will actually be taken.

The Branch History Table records the recent behavior of each branch instruction. Instead of storing target addresses, it stores prediction information indicating whether the branch is likely to be:

- Taken
- Not Taken

Sargantana implements this functionality through the Bimodal Predictor, which uses saturating prediction counters that are continuously updated using previous execution history.

The BHT therefore answers:

"Will this branch be taken?"

Together, the BTB and BHT determine both the destination and the likelihood of branching.

RTL References

- *rtl/datapath/rtl/if_stage_1/rtl/bimodal_predictor.sv*
- *rtl/datapath/rtl/if_stage_1/rtl/branch_predictor.sv*

5.5 Return Address Stack (RAS)

Function calls and returns occur very frequently in software. Predicting return instructions using only the BTB is often inaccurate because a single function may be called from many different locations.

To improve prediction accuracy, Sargantana implements a Return Address Stack (RAS).

Operation:

- CALL instruction → push return address.

- RETURN instruction → pop return address.
- popped address becomes predicted next PC.

This significantly improves prediction accuracy for nested function calls.

The Return Address Stack is implemented as an independent hardware structure inside the Fetch stage.

RTL References

- *rtl/datapath/rtl/if_stage_1/rtl/return_address_stack.sv*

5.6 Prediction Flow

The overall prediction process performed during instruction fetch is summarized below.

1. Current Program Counter enters Fetch-1.
2. Branch Predictor receives the PC.
3. BTB checks whether the PC corresponds to a known branch.
4. BHT predicts whether the branch will be taken.
5. RAS provides return addresses for function returns when required.
6. Predictor selects the next PC.
7. Fetch-2 retrieves instructions from the predicted address.
8. Later execution stages verify the correctness of the prediction.
9. If incorrect, Recovery Logic flushes the pipeline and redirects execution.

This mechanism allows Sargantana to maintain a high instruction throughput while minimizing branch penalties.

RTL References

- *rtl/datapath/rtl/if_stage_1/rtl/if_stage_1.sv*
- *rtl/datapath/rtl/if_stage_1/rtl/branch_predictor.sv*
- *rtl/control_unit/rtl/control_unit.sv*

5.7 Recovery from Mispredictions

A branch prediction is considered complete only after the branch instruction executes. If the prediction matches the actual result, execution proceeds normally. Otherwise, the processor performs a recovery sequence:

- Flush speculative instructions.
- Restore the correct architectural state.

- Update predictor history.
- Redirect the Program Counter.
- Resume instruction fetch.

The recovery logic works together with:

- Instruction Queue
- Rename Tables
- Free Lists
- Graduation List
- Control Unit

to ensure that incorrectly executed speculative instructions never modify the committed architectural state.

RTL References

- *rtl/control_unit/rtl/control_unit.sv*
- *rtl/datapath/rtl/ir_stage/rtl/instruction_queue.sv*
- *rtl/datapath/rtl/ir_stage/rtl/rename_table.sv*
- *rtl/datapath/rtl/ir_stage/rtl/free_list.sv*
- *rtl/datapath/rtl/wb_stage/rtl/graduation_list.sv*

Chapter 6

Decode Stage

6.1 Overview

The Decode stage receives fetched instructions from Fetch-2 and converts them into an internal representation suitable for later pipeline stages.

Its responsibilities include:

- decoding the instruction opcode,
- extracting source and destination registers,
- generating immediate values,
- identifying instruction formats,
- producing control information,
- forwarding decoded instructions to the Rename stage.

This stage forms the interface between instruction fetch and out-of-order execution.

RTL References

- *rtl/datapath/rtl/id_stage/rtl/decoder.sv*
- *rtl/datapath/rtl/id_stage/rtl/immediate.sv*

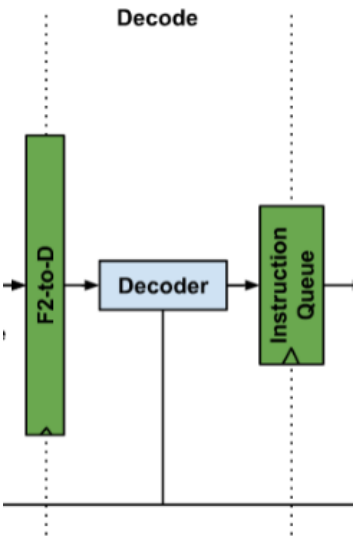

Figure 6.1: Decode stage: F2-to-D → Decoder → Instruction Queue

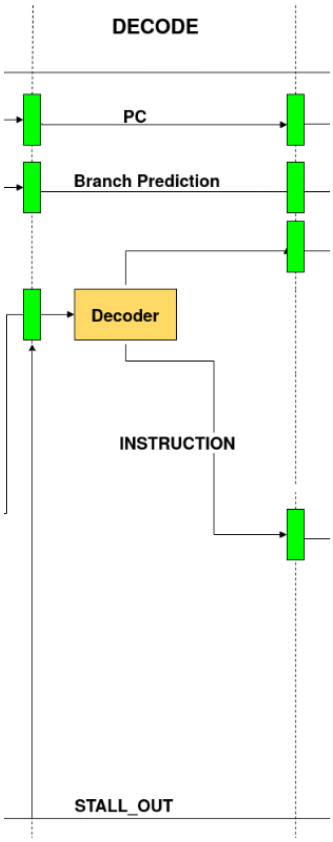

Figure 6.2: Detailed signal view of the Decode stage

6.2 Instruction Decode

The Decoder interprets the binary instruction according to the RISC-V ISA. It identifies:

- instruction format,
- functional unit,
- register operands,
- immediate usage,
- memory operations,
- branch instructions,
- CSR accesses,
- floating-point instructions,
- vector instructions.

The decoded information is forwarded toward Rename.

RTL References

- *rtl/datapath/rtl/id_stage/rtl/decoder.sv*

6.3 Immediate Generation

Many RISC-V instructions contain embedded immediate fields. The Immediate Generator extracts these fields and reconstructs complete signed values according to the instruction format.

Supported formats include:

- I-type
- S-type
- B-type
- U-type
- J-type

These immediates are later consumed by the ALU, Branch Unit, and Address Generation Unit.

RTL References

- *rtl/datapath/rtl/id_stage/rtl/immediate.sv*

6.4 Opcode Decoding

Opcode decoding determines the exact class of instruction. Examples include:

- Arithmetic
- Logical
- Branch
- Jump
- Load
- Store
- Floating Point
- SIMD
- CSR
- System

The Decoder produces internal control signals used by downstream stages to select the appropriate execution unit.

RTL References

- *rtl/datapath/rtl/id_stage/rtl/decoder.sv*

6.5 Control Signal Generation

Based on the decoded instruction, the Decode stage generates the control information required throughout the remainder of the pipeline.

These signals specify:

- destination register validity,
- operand selection,
- ALU operation,
- branch type,
- memory access type,
- CSR operation,
- vector operation,
- floating-point operation.

These control signals accompany the instruction into the Rename stage.

RTL References

- *rtl/datapath/rtl/id_stage/rtl/decoder.sv*
- *rtl/control_unit/rtl/control_unit.sv*

6.6 Output to Rename Stage

After decoding completes, the instruction is forwarded to the Rename stage.

At this point the instruction has:

- decoded opcode,
- source registers,
- destination register,
- immediate value,
- generated control signals.

The Rename stage will subsequently eliminate false register dependencies using register renaming before dispatching instructions toward execution.

RTL References

- *rtl/datapath/rtl/id_stage/rtl/decoder.sv*
- *rtl/datapath/rtl/ir_stage/rtl/rename_table.sv*
- *rtl/datapath/rtl/ir_stage/rtl/instruction_queue.sv*

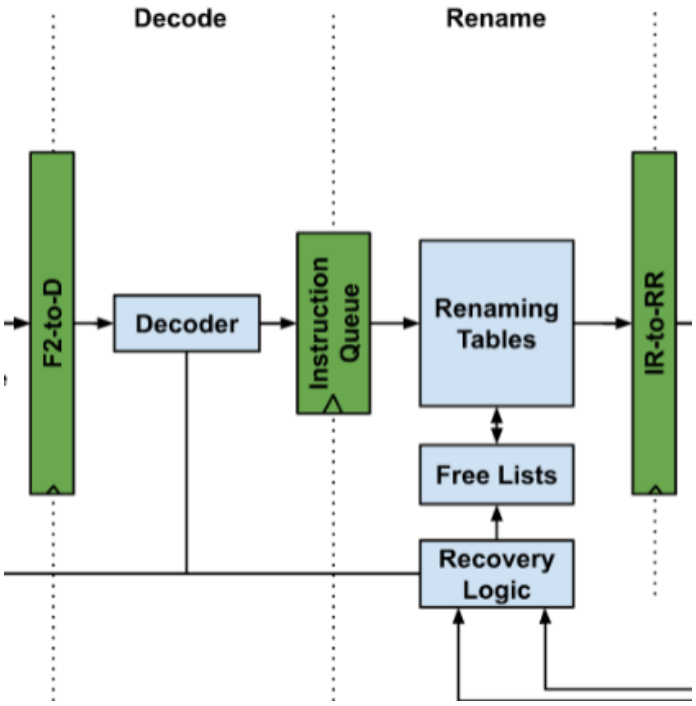

Figure 6.3: Decoded instructions flowing from the Decode stage into the Rename stage

Example (adapted from `doc/specification/decoder_specification`)

The cases are:

- Cycle 1: Invalid input → invalid output.
- Cycle 2: Exception input → exception output and not decoded.
- Cycle 3: Regular case, valid input, not illegal instruction, and signed operation.
- Cycle 4: Regular case again, but it has the power to change the PC.
- Cycle 5: Case with illegal instruction, hence exception valid to 1.
- Cycle 6: Case of the fence with the special signal to 1.

In the following figure we can see all the cases mentioned above.

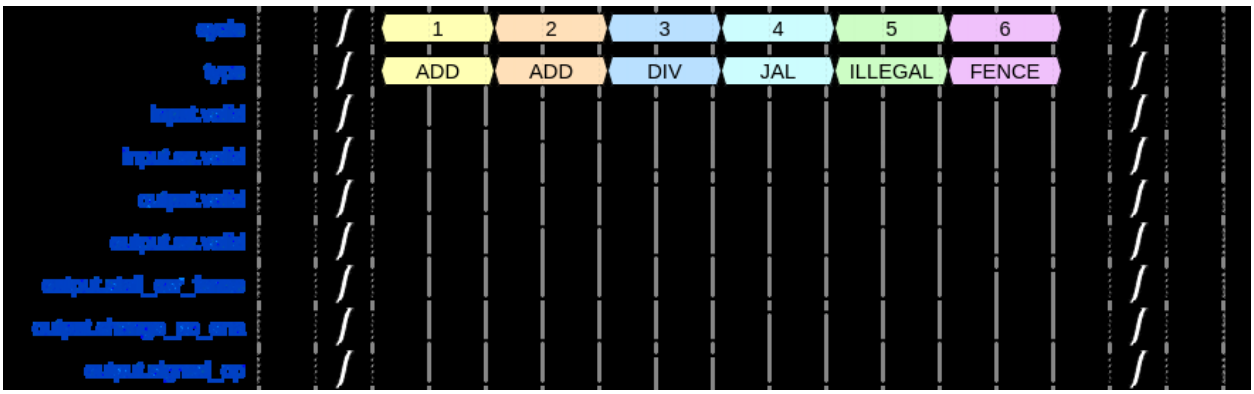

Figure 6.4: Decoder timing waveform illustrating the six example cases

Chapter 7

Register Rename Stage

7.1 Overview

After an instruction has been decoded, it enters the Register Rename (IR) stage. The primary objective of this stage is to eliminate false dependencies caused by reuse of architectural register names.

In the Sargantana processor, the rename stage consists of three major components:

- Register Alias Table (Rename Table)
- Free List
- Instruction Queue

These modules are implemented in:

rtl/datapath/rtl/ir_stage/

Important RTL files include:

RTL References

- *rtl/datapath/rtl/ir_stage/rtl/rename_table.sv*
- *rtl/datapath/rtl/ir_stage/rtl/free_list.sv*
- *rtl/datapath/rtl/ir_stage/rtl/instruction_queue.sv*

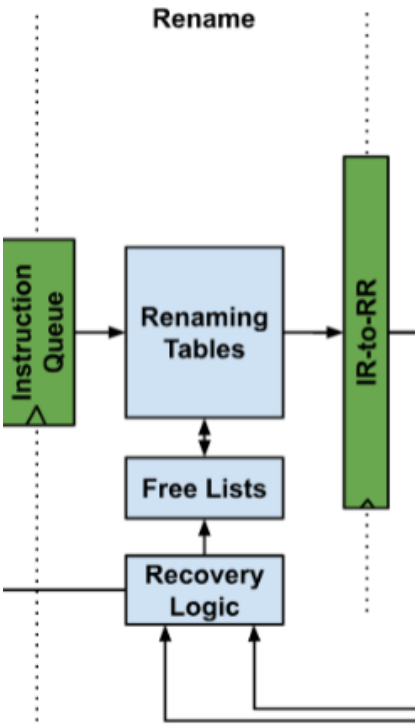

Figure 7.1: Rename stage: Instruction Queue, Renaming Tables, Free Lists, and Recovery Logic

7.2 Register Renaming

Modern superscalar processors allow multiple instructions to execute simultaneously. If two instructions write to the same architectural register, false dependencies arise even when the instructions are logically independent.

Register renaming solves this problem by mapping architectural registers onto a much larger pool of physical registers.

For example:

```
ADD x5, x1, x2
SUB x5, x3, x4
```

Architecturally, both instructions write to `x5`.

Without renaming:

```
ADD -----> x5
SUB -----> x5
```

The second instruction must wait.

With renaming:

```
ADD -----> P17
SUB -----> P22
```

Both instructions can execute independently.

This mapping is maintained inside:

rtl/datapath/rtl/ir_stage/rtl/rename_table.sv

7.3 Register Alias Table (RAT)

The Register Alias Table (RAT), implemented as the Rename Table, stores the current mapping between architectural registers and physical registers.

Rather than reading an architectural register directly, later stages first consult the Rename Table to determine which physical register currently holds the latest value.

Whenever a new destination register is renamed, the table updates its mapping.

The implementation resides in:

rtl/datapath/rtl/ir_stage/rtl/rename_table.sv

The Rename Table therefore becomes the central structure responsible for maintaining correct register mappings throughout speculative execution.

7.4 Physical Register Allocation

Each new destination register requires a free physical register.

Instead of searching all registers every cycle, Sargantana maintains a dedicated Free List.

The Free List keeps track of unused physical registers.

Whenever an instruction requires a destination register, the allocation proceeds as shown below.

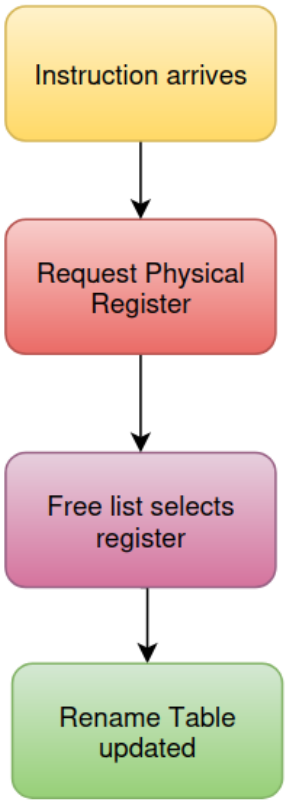

Figure 7.2: Physical register allocation flow: Instruction arrives → Request Physical Register → Free List selects register → Rename Table updated

When an instruction finally commits, its old physical register is returned to the Free List for reuse.

This functionality is implemented in:

rtl/datapath/rtl/ir_stage/rtl/free_list.sv

Together, the Free List and Rename Table enable efficient dynamic register allocation.

7.5 Hazard Elimination

The rename stage primarily eliminates false dependencies:

- Write After Write (WAW)
- Write After Read (WAR)

Because each destination register receives a unique physical register, later instructions no longer overwrite the results of earlier instructions.

Only true data dependencies (RAW) remain.

The renamed instructions are temporarily stored inside the Instruction Queue before being issued to the Register Read stage.

Instruction Queue implementation:

rtl/datapath/rtl/ir_stage/rtl/instruction_queue.sv

The rename stage therefore prepares instructions for high-performance out-of-order execution while maintaining correct architectural state.

Chapter 8

Register Read Stage

8.1 Overview

Following register renaming, instructions enter the Register Read (RR) stage. At this point, all architectural register references have already been converted into physical register identifiers.

The purpose of this stage is to obtain the actual operand values required for execution.

In Sargantana, the Register Read stage is centered around the physical register file.

RTL location:

rtl/datapath/rtl/rr_stage/

Primary implementation:

rtl/datapath/rtl/rr_stage/rtl/regfile.sv

This stage receives renamed instructions from the Rename stage and forwards operand values toward the Execution stage.

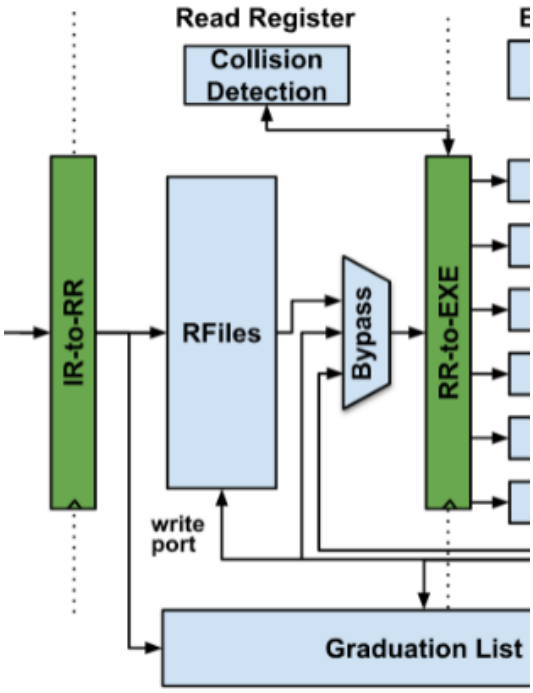

Figure 8.1: Register Read stage: RFiles, Bypass, and Collision Detection

8.2 Register File

The register file stores all physical register values currently available inside the processor. Unlike a simple in-order processor, the register file is accessed using physical register numbers generated during register renaming.

The module is implemented in:

rtl/datapath/rtl/rr_stage/rtl/regfile.sv

The register file supports:

- Reading source operands
- Writing completed results back into physical registers
- Supplying operands to multiple execution units

This module serves as the central storage element for operand values.

8.3 Operand Read

Once physical register identifiers are available, the register file reads the corresponding operand values.

For an instruction:

```
ADD P17, P4, P8
```

the Register Read stage performs:

```
Read P4
Read P8
  |
Operand A
Operand B
  |
Execution Stage
```

This process occurs every cycle for all issued instructions.

The implementation is handled by:

rtl/datapath/rtl/rr_stage/rtl/regfile.sv

8.4 Operand Forwarding

Waiting for every instruction to write back before reading operands would significantly reduce performance. To minimize stalls, recently computed values may be forwarded directly from execution units.

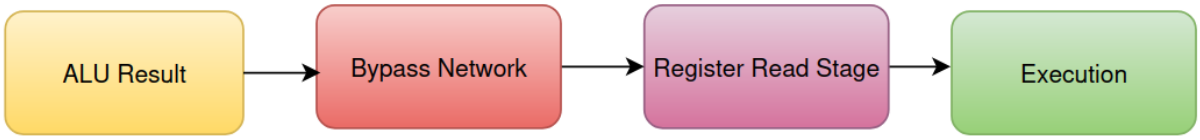

Figure 8.2: Conceptual bypass path: ALU Result → Bypass Network → Register Read Stage → Execution

The pipeline overview shows this Bypass path between the Register Read and Execution stages.

Although the forwarding logic spans multiple execution modules, the Register Read stage is designed to receive these forwarded values whenever available.

8.5 Dependency Resolution

Even after register renaming, true data dependencies (Read After Write) must still be respected.

The Register Read stage ensures that:

- Ready operands are immediately read from the register file.
- Forwarded operands are used whenever newer values exist.
- Instructions with unresolved dependencies wait until their operands become available.

Once all operands are available, the instruction advances to the Execution stage, where arithmetic, logical, memory, floating-point, SIMD, or branch operations are performed.

The interaction between Register Read and Execution is coordinated through:

rtl/datapath/rtl/datapath.sv

and the execution subsystem rooted at:

rtl/datapath/rtl/exe_stage/rtl/exe_stage.sv

This completes the front-end portion of the Sargantana datapath before execution begins.

Chapter 9

Execution Stage

9.1 Overview

The Execution (EXE) stage is responsible for performing the actual computation requested by each instruction. After operands have been read from the physical register file, instructions are dispatched to the appropriate execution unit based on their operation type.

Unlike a simple in-order processor that contains a single ALU, Sargantana contains multiple specialized execution units capable of executing integer, branch, multiplication, division, floating-point, SIMD and memory operations simultaneously.

The entire execution subsystem is implemented under:

rtl/datapath/rtl/exe_stage/

The top-level execution module is:

rtl/datapath/rtl/exe_stage/rtl/exe_stage.sv

This module acts as the dispatcher for all execution units and interfaces with the Load/Store subsystem, scoreboards, writeback logic and pipeline control.

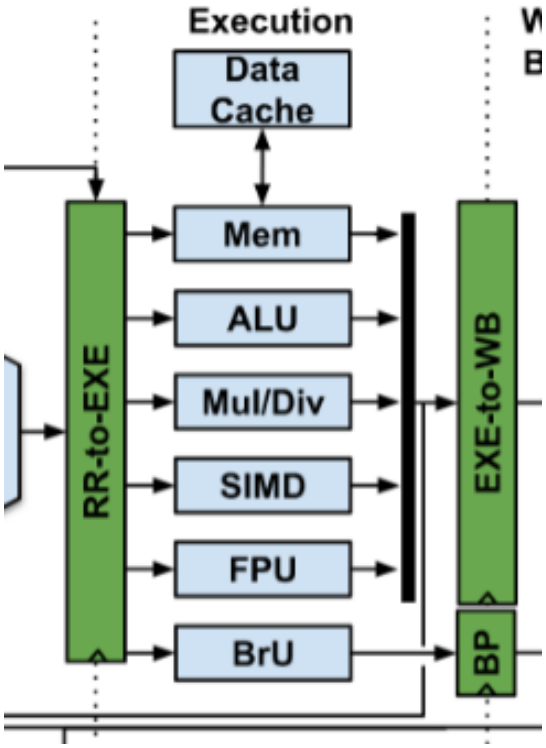

Figure 9.1: Execution stage overview: RR-to-EXE dispatch to Mem, ALU, Mul/Div, SIMD, FPU, and BrU

9.2 Arithmetic Logic Unit (ALU)

The Arithmetic Logic Unit performs all integer arithmetic and logical operations. Instead of implementing every operation inside one large module, Sargantana divides the ALU into several dedicated functional blocks.

The top-level ALU is implemented in:

rtl/datapath/rtl/exe_stage/rtl/alu/alu.sv

Internally, the ALU is divided into the following modules:

RTL File	Function
alu_add.sv	Addition and subtraction
alu_logic.sv	AND, OR, XOR operations
alu_shift.sv	Logical and arithmetic shifts
alu_cmp.sv	Comparison instructions (SLT, SLTU, etc.)
alu_count_pop.sv	Population count instructions
zero_counter/alu_count_zeros.sv	Leading/trailing zero count operations

This modular implementation simplifies verification while allowing new ALU extensions to be integrated independently.

9.3 Branch Unit

Branch instructions require a different execution path than arithmetic instructions because they determine the next program counter value.

The Branch Unit performs:

- Conditional branch evaluation
- Branch target calculation
- Branch taken/not-taken decision
- Branch misprediction recovery support

RTL implementation:

rtl/datapath/rtl/exe_stage/rtl/branch_unit.sv

Although branch prediction occurs during the fetch stage, the Branch Unit provides the final verification of branch outcomes during execution. Any prediction mismatch is reported back to the pipeline so that incorrect speculative instructions can be flushed.

9.4 Multiplier Unit

Integer multiplication operations are handled by a dedicated multiplier unit.

RTL implementation:

rtl/datapath/rtl/exe_stage/rtl/mul_unit.sv

Separating multiplication from the ALU allows multiplication instructions to execute independently without increasing the critical path of simple arithmetic operations.

The multiplier supports RISC-V integer multiplication instructions while integrating with the pipeline's scoreboard and scheduling logic.

9.5 Divider Unit

Division operations typically require multiple clock cycles and therefore utilize a dedicated hardware divider.

The Divider subsystem consists of:

RTL References

- *rtl/datapath/rtl/exe_stage/rtl/div_unit.sv*
- *rtl/datapath/rtl/exe_stage/rtl/div_4bits.sv*

The divider performs integer division and remainder calculations while allowing the processor to continue issuing independent instructions. The scoreboard tracks outstanding division operations until completion.

9.6 Memory Unit

Load and store instructions are processed by the Memory Unit, which interfaces directly with the Load/Store Queue and the cache subsystem.

Primary RTL modules include:

RTL References

- *rtl/datapath/rtl/exe_stage/rtl/mem_unit.sv*
- *rtl/datapath/rtl/exe_stage/rtl/load_store_queue.sv*
- *rtl/datapath/rtl/exe_stage/rtl/store_buffer.sv*
- *rtl/datapath/rtl/exe_stage/rtl/vagu.sv*

The responsibilities of these modules include:

- Effective address calculation
- Load request generation
- Store buffering
- Memory dependency handling
- Interface with the memory hierarchy

The dedicated Address Generation Unit (*vagu.sv*) computes effective memory addresses before memory accesses are issued.

9.7 Execution Pipeline Flow

Once an instruction reaches the Execution stage, it is dispatched to the appropriate functional unit according to its opcode.

The execution flow can be summarized as shown in Figure 9.2.

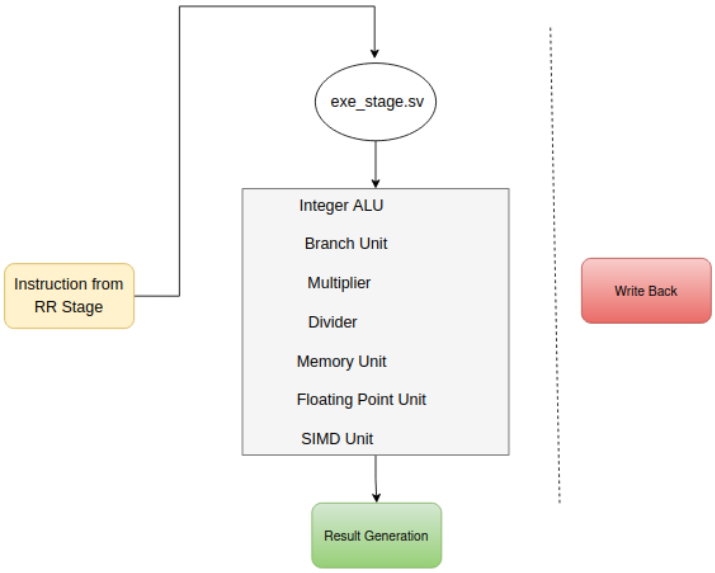

Figure 9.2: Execution pipeline flow: dispatch through `exe_stage.sv` to the functional units and result generation

The execution stage also integrates several auxiliary modules that coordinate long-latency operations:

RTL References

- *pending_mem_req_queue.sv*
- *pending_fp_ops_queue.sv*
- *score_board_scalar.sv*
- *score_board_simd.sv*

These modules allow Sargantana to support out-of-order execution while maintaining correct instruction dependencies.

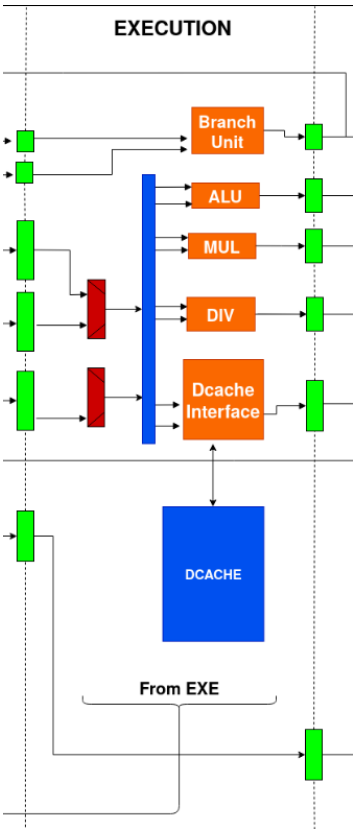

Figure 9.3: Detailed execution-stage datapath, including the D-cache interface and auxiliary queues

Note: it was refined later on, I guess, because the current pipeline is a bit more advanced!

Chapter 10

Memory Subsystem

10.1 Overview

The Memory Subsystem manages all load and store operations generated by the processor. Rather than accessing memory directly from the execution stage, Sargantana uses dedicated hardware structures to support efficient memory scheduling, dependency tracking and store buffering.

The primary RTL modules involved in memory operations are:

RTL References

- *rtl/datapath/rtl/exe_stage/rtl/mem_unit.sv*
- *rtl/datapath/rtl/exe_stage/rtl/load_store_queue.sv*
- *rtl/datapath/rtl/exe_stage/rtl/store_buffer.sv*
- *rtl/datapath/rtl/exe_stage/rtl/vagu.sv*
- *rtl/datapath/rtl/interface_csr/rtl/csr_interface.sv*

These modules collectively implement the processor's memory execution pipeline.

10.2 Load Operations

When a load instruction reaches the Execution stage, the effective address is first computed by the Address Generation Unit (*vagu.sv*). The request is then inserted into the Load/Store Queue, which manages pending memory operations.

The Memory Unit (*mem_unit.sv*) communicates with the cache hierarchy and retrieves the requested data. Once the data becomes available, it is forwarded toward the Writeback stage, where it updates the corresponding physical register.

This separation allows multiple outstanding memory requests to be tracked without stalling the entire pipeline.

10.3 Store Operations

Store instructions follow a similar flow but differ in that they write data to memory rather than returning a result to the register file.

Before a store is committed to memory, it is temporarily held in the Store Buffer:

rtl/datapath/rtl/exe_stage/rtl/store_buffer.sv

The Store Buffer allows execution to continue while pending store operations are safely queued. Once it is safe to update memory, buffered store entries are committed in program order.

10.4 Address Generation

Effective memory addresses are generated by the dedicated Address Generation Unit.

RTL implementation:

rtl/datapath/rtl/exe_stage/rtl/vagu.sv

The Address Generation Unit combines the base register value with the instruction offset to compute the final virtual address used by both load and store operations.

Separating address generation into its own module improves modularity and enables future support for more advanced addressing modes.

10.5 Cache Interface

The Memory Unit serves as the interface between the execution pipeline and the processor's cache hierarchy. It issues memory requests generated by the Load/Store Queue and receives the corresponding responses.

The cache interface logic is primarily coordinated by:

rtl/datapath/rtl/exe_stage/rtl/mem_unit.sv

and integrated into the top-level datapath through:

rtl/datapath/rtl/datapath.sv

This organization isolates cache communication from the rest of the execution logic while allowing memory latency to be managed independently.

10.6 Memory Ordering

Correct program execution requires that memory operations observe architectural ordering rules. Sargantana enforces this through the interaction of the Load/Store Queue, Store Buffer and scoreboard structures.

The relevant RTL modules include:

RTL References

- *load_store_queue.sv*
- *store_buffer.sv*
- *pending_mem_req_queue.sv*
- *score_board_scalar.sv*

Together, these modules track outstanding memory requests, preserve dependencies between loads and stores, and ensure that memory operations are completed in a correct and consistent order before instructions are retired.

Chapter 11

Writeback Stage

11.1 Overview

The Writeback (WB) stage is responsible for collecting the completed results produced by the execution units and making them available for architectural retirement. Unlike an in-order processor where results are written directly into the register file, Sargantana employs an out-of-order backend. Consequently, completed execution results are first gathered and associated with their corresponding physical registers before the instruction becomes eligible for retirement.

The writeback stage therefore acts as the interface between execution and commit. It receives completion notifications from multiple functional units such as the ALU, multiplier, divider, floating-point unit, SIMD unit, and memory subsystem. Once a result is available, the stage updates the internal completion status so that dependent instructions may proceed and the commit stage can eventually retire the instruction.

Unlike the execution stage, the writeback logic mainly manages completed instruction information rather than performing arithmetic computations.

RTL References

- *rtl/datapath/rtl/wb_stage/rtl/graduation_list.sv*
- *rtl/datapath/rtl/datapath.sv*

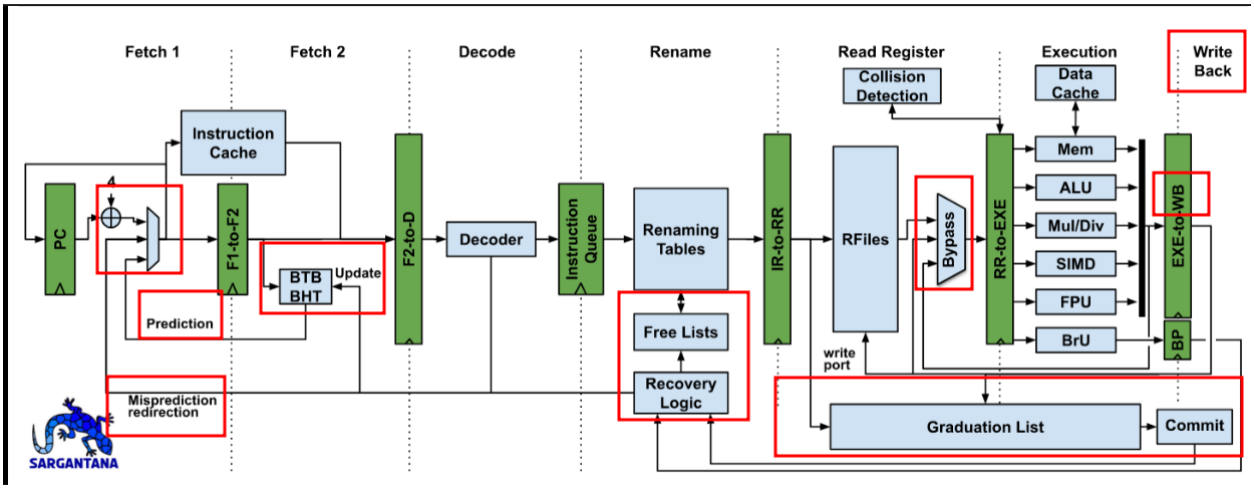

Figure 11.1: Writeback stage in context: results flow from the execution units into the Graduation List and Commit

11.2 Result Collection

Multiple execution units operate simultaneously and may finish execution at different clock cycles. The writeback stage therefore collects completed results from all available execution pipelines.

Typical producers include:

- Integer ALU
- Branch Unit
- Multiplier
- Divider
- Memory Unit
- Floating Point Unit
- SIMD Unit

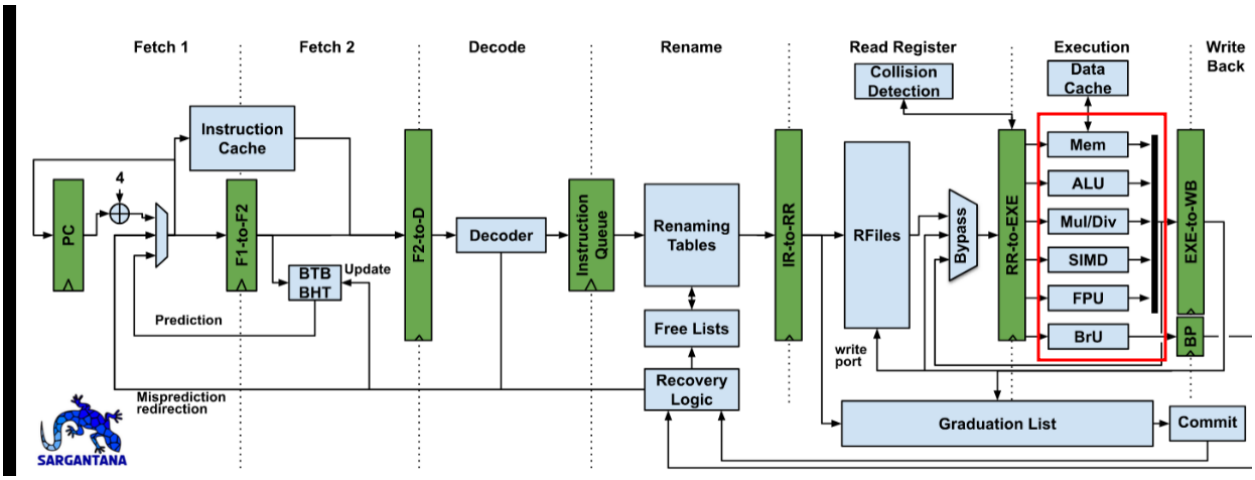

Figure 11.2: Result collection: completed results from the highlighted execution units feed the writeback stage

Each completed instruction provides:

- Destination physical register
- Computed result
- Completion indication
- Exception information (if generated)

Rather than immediately committing the instruction, the writeback stage records that the instruction has completed successfully.

The RTL implements this functionality through the **Graduation List**, which keeps track of instructions that have completed execution and are waiting for architectural retirement.

RTL References

- *rtl/datapath/rtl/wb_stage/rtl/graduation_list.sv*

11.3 Register Update

Because Sargantana uses physical register renaming, execution results are written into the allocated physical register instead of directly updating the architectural register file.

The sequence is:

1. Register Rename allocates a physical register.
2. Execution computes the instruction result.
3. Writeback stores the value into the physical register.
4. Commit later updates the architectural mapping.

This separation allows speculative instructions to execute safely without modifying the visible architectural state.

The Graduation List maintains the completion status for each instruction until the Commit stage verifies that the instruction can safely retire.

This mechanism supports:

- speculative execution
- precise exceptions
- rollback after branch misprediction
- correct in-order retirement

RTL References

- *rtl/datapath/rtl/ir_stage/rtl/rename_table.sv*
- *rtl/datapath/rtl/rr_stage/rtl/regfile.sv*
- *rtl/datapath/rtl/wb_stage/rtl/graduation_list.sv*

11.4 Exception Information

Besides carrying execution results, the writeback stage also forwards exception information generated during execution.

Examples include:

- Illegal instruction
- Load or Store access fault
- Misaligned memory access
- Arithmetic exceptions
- Floating-point exceptions

Instead of immediately servicing an exception, the writeback stage records the exception status alongside the completed instruction.

The Commit stage later checks whether the faulting instruction is the oldest instruction in the machine. Only then is the architectural state updated and the exception is delivered, guaranteeing precise exception handling.

This approach ensures that younger speculative instructions never modify architectural state before older instructions have safely completed.

RTL References

- *rtl/datapath/rtl/wb_stage/rtl/graduation_list.sv*
- *rtl/control_unit/rtl/control_unit.sv*

Chapter 12

Commit Stage

12.1 Overview

The Commit stage is the final stage of the Sargantana pipeline and is responsible for updating the architectural state in program order. Although instructions may execute out of order, they must always retire sequentially to preserve the behavior defined by the RISC-V ISA.

Only instructions that have successfully completed execution and whose older instructions have already retired are allowed to commit. This guarantees precise exceptions and ensures that speculative execution remains invisible to software.

The commit stage therefore represents the point at which speculative results become architecturally permanent.

RTL References

- *rtl/datapath/rtl/wb_stage/rtl/graduation_list.sv*
- *rtl/control_unit/rtl/control_unit.sv*
- *rtl/datapath/rtl/datapath.sv*

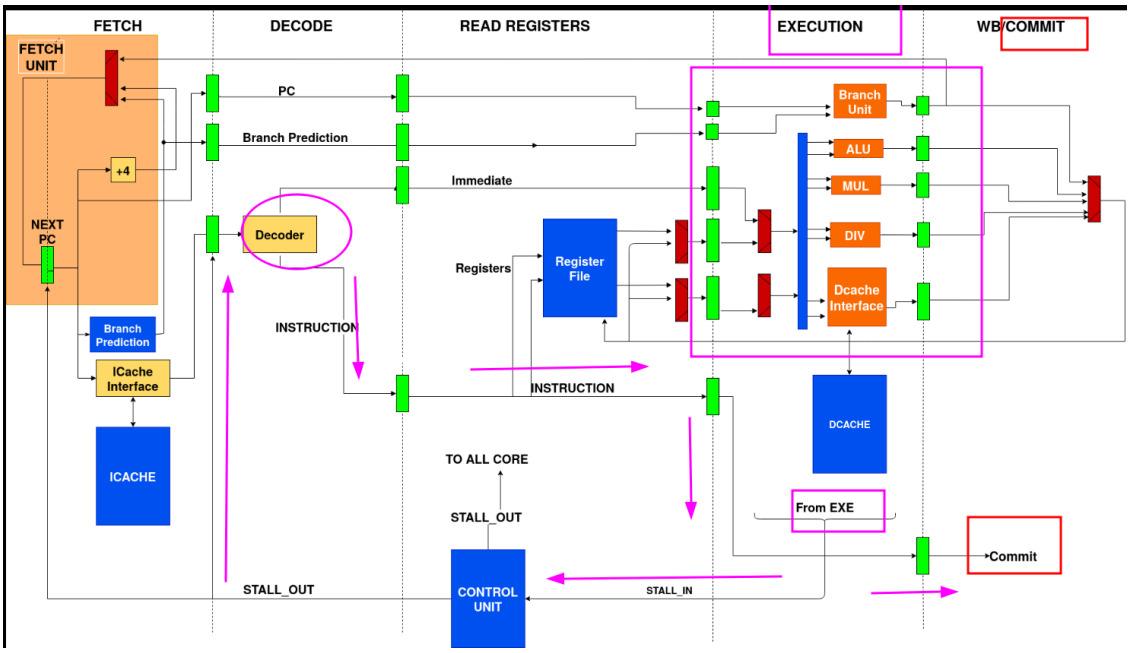

Figure 12.1: Commit stage in context, highlighting the decode-to-commit control path

12.2 Reorder Buffer (ROB)

Although the repository does not contain a standalone module named *rob.sv*, the reorder/retirement functionality is implemented through the **Graduation List**, together with the rename structures and control logic.

The Graduation List maintains the order of all in-flight instructions from dispatch until retirement.

For each instruction it tracks information such as:

- destination physical register
- completion status
- exception status
- branch information
- retirement eligibility

When execution finishes, the corresponding entry is marked as completed. The Commit stage continuously checks the oldest valid entry. If the instruction has completed without exceptions, it is retired.

Thus, the Graduation List performs the architectural role traditionally associated with a Reorder Buffer (ROB).

RTL References

- *rtl/datapath/rtl/wb_stage/rtl/graduation_list.sv*
- *rtl/datapath/rtl/ir_stage/rtl/rename_table.sv*

- *rtl/datapath/rtl/ir_stage/rtl/free_list.sv*

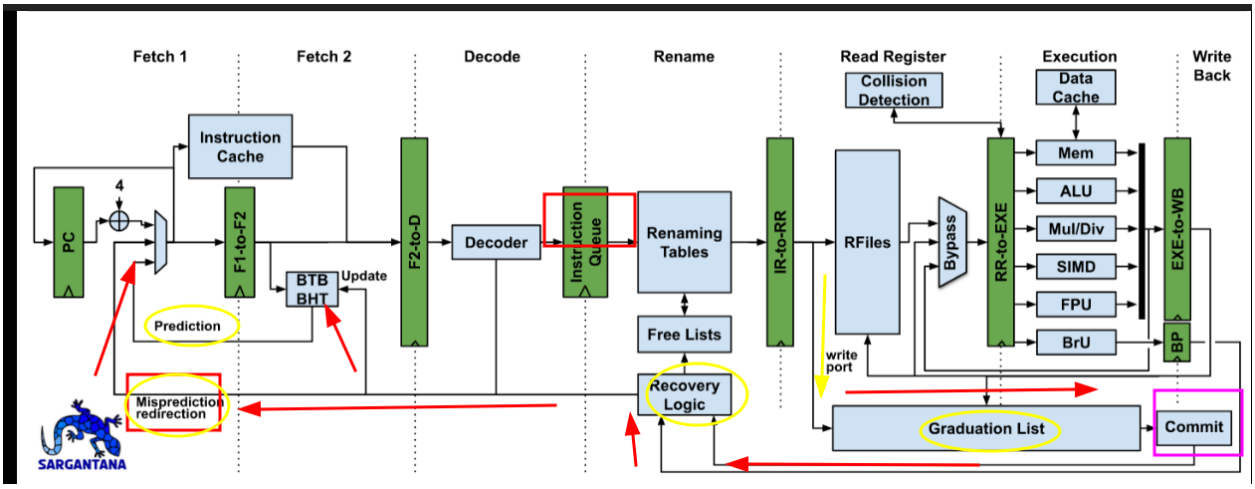

Figure 12.2: Graduation List acting as the Reorder Buffer: recovery and commit signal paths highlighted

12.3 In-Order Retirement

Although execution is highly parallel, retirement always occurs in program order.

For each instruction, the Commit stage performs several checks before retirement:

- Has execution completed?
- Did the instruction raise an exception?
- Is it the oldest instruction?
- Are all previous instructions already retired?

Only after satisfying these conditions are the architectural structures updated.
Retirement includes:

- updating the architectural register mapping
- releasing obsolete physical registers
- committing memory operations
- updating CSR state if required
- removing the instruction from the Graduation List

This mechanism ensures that software observes execution exactly as if instructions had been executed sequentially.

RTL References

- *rtl/datapath/rtl/wb_stage/rtl/graduation_list.sv*
- *rtl/datapath/rtl/ir_stage/rtl/free_list.sv*
- *rtl/datapath/rtl/ir_stage/rtl/rename_table.sv*

12.4 Exception Handling

One of the primary responsibilities of the Commit stage is precise exception handling.

If an instruction reports an exception during execution, retirement stops when that instruction reaches the head of the Graduation List.

The processor then:

- flushes younger speculative instructions
- restores the correct architectural state
- redirects the PC to the exception handler
- updates the required CSRs
- resumes execution from the exception vector

Since no younger instruction has committed yet, software observes a perfectly precise exception. This behavior is one of the major advantages of separating execution from retirement.

RTL References

- *rtl/control_unit/rtl/control_unit.sv*
- *rtl/datapath/rtl/wb_stage/rtl/graduation_list.sv*
- *rtl/datapath/interface_csr/rtl/csr_interface.sv*

12.5 Pipeline Flush and Recovery

Whenever speculation proves incorrect, the Commit stage coordinates recovery.

Typical recovery events include:

- Branch misprediction
- Exceptions
- Interrupts
- System reset
- CSR redirection

Recovery generally involves:

1. stopping retirement,
2. flushing younger instructions,
3. restoring the correct rename state,
4. recovering the free list,
5. redirecting the Program Counter,
6. restarting instruction fetch.

Because retirement is always in-order, recovery restores the processor to a precise architectural state.

RTL References

- *rtl/control_unit/rtl/control_unit.sv*
- *rtl/datapath/rtl/if_stage_1/rtl/branch_predictor.sv*
- *rtl/datapath/rtl/if_stage_1/rtl/if_stage_1.sv*
- *rtl/datapath/rtl/wb_stage/rtl/graduation_list.sv*
- *rtl/datapath/rtl/ir_stage/rtl/rename_table.sv*
- *rtl/datapath/rtl/ir_stage/rtl/free_list.sv*

Chapter 13

Recovery Mechanisms

13.1 Overview

Recovery in DRAC ensures that the processor returns to a precise architectural state whenever speculative execution fails or an exception occurs. Since the processor executes instructions out-of-order, incorrect speculative instructions must be removed while preserving the committed architectural state.

13.2 Branch Misprediction Recovery

This section covers:

- Branch prediction in IF Stage 1
- Actual branch resolution in the Execution Stage (*branch_unit.sv*)
- Detection of incorrect prediction
- Redirecting the Program Counter
- Flushing younger speculative instructions
- Restoring rename mappings and scoreboard entries

RTL References

- *rtl/datapath/rtl/if_stage_1/branch_predictor.sv*
- *rtl/datapath/rtl/if_stage_1/bimodal_predictor.sv*
- *rtl/datapath/rtl/if_stage_1/return_address_stack.sv*
- *rtl/datapath/rtl/exe_stage/branch_unit.sv*
- *rtl/datapath/rtl/ir_stage/rename_table.sv*
- *rtl/datapath/rtl/ir_stage/free_list.sv*
- *rtl/datapath/rtl/exe_stage/score_board_scalar.sv*
- *rtl/datapath/rtl/exe_stage/score_board_simd.sv*
- *rtl/control_unit/rtl/control_unit.sv*

13.3 Exception Recovery

This section covers:

- Arithmetic exceptions
- Illegal instructions
- Memory exceptions
- CSR exceptions
- Exception propagation
- Precise exception model

RTL References

- *rtl/csr/*
- *rtl/datapath/rtl/interface_csr/csr_interface.sv*
- *rtl/control_unit/*

13.4 Pipeline Flush Mechanism

This section covers:

- Flush signal generation
- Invalidating pipeline entries
- Clearing issue queues
- Clearing scoreboards
- Clearing pending memory requests
- Cancelling speculative instructions

RTL References

- *rtl/control_unit/*
- *rtl/datapath/rtl/exe_stage/rtl/pending_mem_req_queue.sv*
- *rtl/datapath/rtl/exe_stage/rtl/pending_fp_ops_queue.sv*
- *rtl/datapath/rtl/exe_stage/rtl/pending_vfp_ops_queue.sv*
- *rtl/datapath/rtl/exe_stage/rtl/load_store_queue.sv*
- *rtl/datapath/rtl/exe_stage/rtl/score_board_scalar.sv*
- *rtl/datapath/rtl/exe_stage/rtl/score_board_simd.sv*

13.5 Restarting Instruction Execution

This section covers:

- Redirect PC
- Restore rename state
- Resume fetch
- Continue normal execution

RTL References

- *rtl/datapath/rtl/if_stage_1/*
- *rtl/datapath/rtl/ir_stage/*
- *rtl/control_unit/*

Appendix A

CRUX of RTL Files and Status

Chapter	RTL Modules / Files	Status
Chapter 9. Execution Stage	*exe_stage.sv*, *alu.sv*, *alu_add.sv*, *alu_logic.sv*, *alu_shift.sv*, *alu_cmp.sv*, *branch_unit.sv*, *mul_unit.sv*, *div_unit.sv*, *mem_unit.sv*, *vagu.sv*, *load_store_queue.sv*, *store_buffer.sv*, *score_board_scalar.sv*, *score_board_simd.sv*, *pending_mem_req_queue.sv*, *pending_fp_ops_queue.sv*	✓ Fully supported
Chapter 10. Memory Subsystem	*mem_unit.sv*, *vagu.sv*, *load_store_queue.sv*, *store_buffer.sv*, *pending_mem_req_queue.sv*	✓ Mostly supported
Chapter 11. Write-back Stage	*wb_stage/rtl/graduation_list.sv*, completion signals coming from execution units	△ Partially supported
Chapter 12. Commit Stage	*wb_stage/rtl/graduation_list.sv*, *ir_stage/rtl/rename_table.sv*, *free_list.sv*, scoreboard modules	△ Commit logic exists but there is no standalone ROB module
Chapter 13. Recovery Mechanisms	*branch_unit.sv*, *branch_predictor.sv*, *return_address_stack.sv*, *control_unit.sv*, rename/free-list recovery logic	△ Supported through distributed logic rather than one recovery module

Legend:

✓ Complete RTL coverage located and matched to the documentation.

△ Functionality exists but is distributed across several modules rather than a single dedicated file.

Appendix B

Abbreviations and Acronyms

Abbr.	Full Form	Description
ALU	Arithmetic Logic Unit	Executes integer arithmetic and logical operations.
AGU	Address Generation Unit	Computes effective addresses for memory instructions.
BTB	Branch Target Buffer	Stores predicted branch target addresses.
BHT	Branch History Table	Stores branch prediction history.
CSR	Control and Status Register	Special-purpose processor registers used for control and exception handling.
CFI	Control-Flow Integrity	Security mechanism preventing illegal control-flow transfers.
CPI	Cycles Per Instruction	Average execution cycles required per instruction.
EX	Execute Stage	Pipeline stage where arithmetic, logical, and branch operations are executed.
FP	Floating Point	Arithmetic performed on floating-point numbers.
FPU	Floating Point Unit	Dedicated execution unit for floating-point instructions.
FSM	Finite State Machine	Hardware state machine controlling logic behavior.
GHR	Global History Register	Stores global branch history for branch prediction.
IF	Instruction Fetch	Pipeline stage responsible for fetching instructions.
ID	Instruction Decode	Pipeline stage where instructions are decoded.
IQ	Instruction Queue	Buffer storing renamed instructions awaiting execution.
IR	Instruction Rename	Pipeline stage performing register renaming.
ISA	Instruction Set Architecture	Software-visible processor architecture.

Abbr.	Full Form	Description
LSQ	Load Store Queue	Tracks pending memory operations while preserving memory ordering.
LPAD	Landing Pad Instruction	Zicfilp instruction used to validate indirect branch targets.
MEM	Memory Stage	Pipeline stage responsible for memory accesses.
OoO	Out-of-Order	Execution model allowing instructions to execute before older instructions when dependencies permit.
PC	Program Counter	Address of the current instruction.
PE	Processing Element	Basic computational element, typically used in vector or matrix accelerators.
RAT	Register Alias Table	Maps architectural registers to physical registers during renaming.
ROB	Reorder Buffer	Tracks in-flight instructions and commits them in program order.
RR	Register Read	Pipeline stage where source operands are read.
RTL	Register Transfer Level	Hardware description abstraction used in Verilog/SystemVerilog.
RAS	Return Address Stack	Hardware stack predicting return instruction targets.
SIMD	Single Instruction Multiple Data	Parallel execution model processing multiple data elements simultaneously.
SQ	Store Queue	Queue buffering store operations before retirement.
SS	Shadow Stack	Protected stack used by Zicfiss to validate return addresses.
SV	SystemVerilog	Hardware Description Language used for RTL implementation.
TB	Testbench	Verification environment for RTL simulation.
WB	Writeback	Pipeline stage where execution results are written back.
XLEN	Integer Register Width	Width of the architectural integer registers (e.g., 32-bit or 64-bit).

Appendix C

Pipeline Stage Abbreviations

Stage	Meaning
IF1	Instruction Fetch Stage 1
IF2	Instruction Fetch Stage 2
ID	Instruction Decode
IR	Instruction Rename
RR	Register Read
EX	Execute
MEM	Memory Access
WB	Writeback
Commit	Instruction Retirement / Graduation

Appendix D

RTL Module Abbreviations

Module	Description
datapath.sv	Top-level datapath implementation
control_unit.sv	Processor control logic
if_stage_1.sv	First instruction fetch stage
if_stage_2.sv	Second instruction fetch stage
decoder.sv	Instruction decoder
immediate.sv	Immediate value generator
rename_table.sv	Register alias table
free_list.sv	Physical register allocator
instruction_queue.sv	Issue queue
regfile.sv	Physical register file
exe_stage.sv	Top-level execution stage
branch_unit.sv	Branch execution logic
alu.sv	Arithmetic Logic Unit
div_unit.sv	Integer divider
mul_unit.sv	Integer multiplier
mem_unit.sv	Memory execution unit
load_store_queue.sv	Load/Store Queue
pending_mem_req_queue.sv	Pending memory request tracker
pending_fp_ops_queue.sv	Pending floating-point operations
score_board_scalar.sv	Scalar dependency scoreboard
score_board_simd.sv	SIMD dependency scoreboard
graduation_list.sv	Instruction commit list
csr_interface.sv	CSR interface logic

Appendix E

RISC-V Extensions Referenced

Extension	Description
RV64	64-bit RISC-V base ISA
M	Integer Multiply/Divide Extension
A	Atomic Extension
F	Single-Precision Floating Point
D	Double-Precision Floating Point
C	Compressed Instructions
V	Vector Extension
Zfa	Additional Floating-Point Instructions
Zfh	Half-Precision Floating Point
Zvfhmin	Minimal Half-Precision Vector Extension
Zicfilp	Control-Flow Integrity Landing Pad Extension
Zicfiss	Control-Flow Integrity Shadow Stack Extension

Appendix F

Execution Unit Components

Component	Function
ALU	Integer arithmetic and logic
Branch Unit	Branch comparison and target computation
Divider	Integer division operations
Multiplier	Integer multiplication operations
Memory Unit	Load and store execution
SIMD Unit	Vector arithmetic execution
FPU	Floating-point execution
AGU	Effective address generation
Scoreboard	Dependency tracking
Pending Queues	Track unfinished operations